CS352 PA3

AST_EMIT(ASTExprStmt) -> call emitIR(ctx) of mExpr object and assign to retVal.

AST_EMIT(ASTCompoundStmt) -> loop through mDecl list and mStmt list. For mDecl emit every pointer. For mStmt, only emit if ctx.mBlock is not terminated. Otherwise break the loop.

AST_EMIT(ASTConstantExpr) -> create a variable to hold the llvm::type. If mType yields a char type then assign getInt8Ty value to the type. Otherwise assign getInt32Ty. Then make is constant using ConstantInt::get(type, mValue) and assign it to retVal.

AST_EMIT(ASTReturnStmt) -> if mExpr is not nullptr then emit and save the returned value. Create a build block and write a return of the value. Otherwise create a build block and return a void value.

AST_EMIT(ASTBinaryMathOp) -> emit the LHS and RHS and save both values. Create the IRBuilder<> build(ctx.mBlock) object. Create a switch on mOp to pick the appropriate create function for the math operation. Then return the instruction

AST_EMIT(ASTBinaryCmpOp) -> same logic as ASTBinaryMathOp but we need to convert the 1 bit bool retVal into a 32 bit integer before returning it. This is done by calling build.CreateZext() on the retVal value and storing it inside retVal.

void SymbolTable::ScopeTable::emitIR(CodeContext& ctx) -> sav ethe old address using ident->getAddress(). Then get its type. After this we have to create a space in the stack and point the identifier to this space. If the parameter exists, then we copy the parameters.

llvm::Value* Identifier::readFrom(CodeContext& ctx) noexcept -> create a builder on ctx.mBlock and call build.CreateLoad() on it. Then assign the value to retVal.

void Identifier::writeTo(CodeContext& ctx, llvm::Value* value) noexcept -> create a builder on ctx.mBlock and call build.CreateStore() on it

AST_EMIT(ASTIncExpr) -> read current value using mIdent.readFrom(ctx) and create a builder. Add 1 using build.CreateAdd(), passing the value itself and 1 as a constant int as parameters. Then write it back using mIdent.writeTo(ctx, val) and return the new value.

AST_EMIT(ASTDecExpr) -> exact same logic as ASTIncExpr but with CreateSub instead of CreateAdd.

AST_EMIT(ASTNotExpr) -> emit mExpr and store the value. Create a builder and then compare if the value is equal to zero using build.CreateICmpEQ(). Store in retVal and zero extend to a 32 bit integer.

AST_EMIT(ASTAssignStmt) -> emit mExpr and then call mIdent.writeTo() on that value.

llvm::Value* ASTArraySub::emitLinearIndex(CodeContext& ctx) noexcept -> iterate through all exprs in mExprs and append them to a vector. Create a builder and then assign the first element of the vector to retVal since it is the first value. Then we use another for loop to iterate over the size of the vector starting from 1 (after the first element) so we can make the extent constant, multiply retVal by the extent and then add the current index of the vector to retVal. We can then return retVal

AST_EMIT(ASTDecl)::emitInitializerList(ctx, init, dimIdx, base) -> this function is recursive. dimIdx is the dimension the current list describes, and base is the flat slot where the list starts. If dimIdx is 0, get the array address using mIdent.readFrom(ctx), work out the size in bytes (getArrayCount() * 4 for an int array), and clear the whole array using build.CreateMemSet() with an i8 zero, so any element the list doesn't supply is zero. Only the top call does this, otherwise nested calls would erase rows already written. Then, in every call, loop through init->getElems() with a counter. If an element is a nested list (checked with dynamic_pointer_cast<ASTArrayInitExpr>), call emitInitializerList on it with dimIdx + 1 and base + counter * mIdent.getStride(dimIdx), which is the first slot of that row. Otherwise, emit the element, create a builder, and truncate the value to i8 if the array is a char array (the value's type differs from llvmElementType). Then get the element's address using CreateInBoundsGEP from the array address with a constant index of base + counter, and store the value there with CreateStore. Increment the counter after every element, nested or not

AST_EMIT(ASTWhileStmt) -> create three blocks: while.cond, while.body and while.end. Create a builder on the current block and branch unconditionally to while.cond with CreateBr. Set ctx.mBlock to while.cond and emit mExpr. Create a new builder on ctx.mBlock, convert the condition to a 1 bit bool with CreateICmpNE against ctx.mZero, then call CreateCondBr to jump to while.body when true or while.end when false. Set ctx.mBlock to while.body and emit mLoopStmt. If ctx.mBlock has no terminator then create a builder on ctx.mBlock and branch back to while.cond so the condition is checked again. Use ctx.mBlock instead of while.body because a nested statement may have moved the tracker. Each builder writes to only one block, so a new builder is

created every time the cursor moves. Set ctx.mBlock to while.end so the statements after the loop are emitted there, and return nullptr

AST_EMIT(ASTIfStmt) -> create the if.then and if.end blocks. If mElseStmt is not nullptr, also create an if.else block. Keep a falseTarget pointer that starts as if.end and is changed to if.else when there is an else, so the condition always knows where to jump when it's false. Emit mExpr in the current block and then create a builder on ctx.mBlock, convert the value to a 1bit bool with CreateICmpNE against ctx.mZero, and call CreateCondBr to jump to if.then when true or falseTarget when false. Set ctx.mBlock to if.then and emit mThenStmt. If ctx.mBlock has no terminator then create a builder on ctx.mBlock and branch to if.end. Use ctx.mBlock rather than if.then because a nested statement may have moved the cursor. If there is an else, do the same for if.else: set ctx.mBlock to it, emit mElseStmt, and add the same guarded branch to if.end. Then set ctx.mBlock to if.end so the statements after the if are emitted there, and return nullptr

AST_EMIT(ASTForStmt) -> create four blocks: for.cond, for.body, for.step and for.end. If minitStmt is not nullptr, emit it in the current block because init only runs once at loop start. Then create a builder on ctx.mBlock and branch to for.cond. Set ctx.mBlock to for.cond and if mcondExpr is not nullptr, emit it and save the value. Create a builder on ctx.mBlock. If the condition value is not nullptr, convert it to a 1 bit bool with CreateICmpNE against ctx.mZero and call CreateCondBr to jump to for.body when true or for.end when false. If there is no condition (eg: for(;;)), branch straight to for.body, since an empty condition means loop forever. Set ctx.mBlock to for.body and emit mbodyStmt. If ctx.mBlock has no terminator, branch to for.step, not for.cond, so the step always runs before the condition is checked again. Set ctx.mBlock to for.step, emit mstepStmt if it is not nullptr and if ctx.mBlock has no terminator then branch back to for.cond. The step has its own block so that continue can jump to it later. Then set ctx.mBlock to for.end and return nullptr

AST_EMIT(ASTDoWhileStmt) -> almost same exact logic as ASTWhileStmt. Create 3 blocks for dowhile.cond, dowhile.body and downhill.end. Just need to reorder the blocks so that body comes before condition and the body block is the entry point.

AST_EMIT(ASTBreakStmt) -> create a builder on ctx.mBlock and branch unconditionally to ctx.mBreakBlocks.top() using CreateBr. The break node doesn't know which loop it is in so the enclosing loop pushes its end block onto mBreakBlocks before emitting its body and pops it afterwards. The top of the stack is always the innermost loop's end, so nested loops work automatically. The branch terminates the current block, so ASTCompoundStmt stops emitting any statements after the break. The if/loop skips adding its own branch because the block already has a terminator

AST_EMIT(ASTContinueStmt) -> same as ASTBreakStmt but branches to ctx.mContinueBlocks.top(). The target depends on the loop: while and do-while push their cond block, so ‘continue’ rechecks the condition, whereas ‘for’ pushes its step block, so ‘continue’ runs the step (e.g. ++i) before checking the condition again. Pushing the cond block for a for loop would skip the step and cause an infinite loop. A switch does not push onto mContinueBlocks, so a continue inside a switch will reach the outer loop

NOTE: while, doWhile and for were all update in accordance to the break and continue. Loop end block was pushed onto mBreakBlocks and loop condition block was pushed onto mContinueBlocks. Both were then popped out of the stack after to allow later break/continue statements to work without interference.

AST_EMIT(ASTSwitchStmt) -> emit mExpr and save the condition value. Loop through case_default_stmts and create one block per entry in source order storing them in a vector so blocks[i] belongs to case_default_stmts[i]. Use dynamic_pointer_cast<ASTDefaultStmt> to spot the default: its block is named switch.default and saved in defaultBlock, and every other entry gets a switch.case block. Then create the switch.end block. Create a builder on ctx.mBlock and call CreateSwitch with the condition value, a default destination. defaultBlock if there is one, else switch.end and the number of entries as a size hint, saving the returned SwitchInst. Loop through the entries by index and, for each one that is an ASTCaseStmt, emit its getExpr() and add it to the switch with sw->addCase, using llvm::cast<ConstantInt> to convert the emitted value, and blocks[i] as the target. Push switch.end onto ctx.mBreakBlocks. Then loop through the entries again: set ctx.mBlock to blocks[i] and emit the entry. If ctx.mBlock has no terminator, branch to the next case's block. Walking in source order means a default in the middle is handled the same way. After the loop, pop ctx.mBreakBlocks and set ctx.mBlock to switch.end, then return nullptr.